

Topic 3: Divide and Conquer

Design and Analysis of Algorithms Laboratory

Exercise 3.1: Find Maximum and Minimum using Divide and Conquer

Aim:

To find both the maximum and minimum elements in an array using the divide-and-conquer tournament method with minimum comparisons ($3n/2 - 2$).

Python Code:

```
def find_min_max_dc(arr, low, high):
    """
    Divide and Conquer Min-Max.
    Number of comparisons:  $\sim 3n/2 - 2$ . Time:  $O(n)$ , Space:  $O(\log n)$ .
    """
    if low == high:
        return arr[low], arr[low]
    if high == low + 1:
        if arr[low] < arr[high]:
            return arr[low], arr[high]
        else:
            return arr[high], arr[low]

    mid = (low + high) // 2
    min1, max1 = find_min_max_dc(arr, low, mid)
    min2, max2 = find_min_max_dc(arr, mid + 1, high)
    return min(min1, min2), max(max1, max2)

# Test Cases
tcs = [
    [5, 7, 3, 4, 9, 12, 6, 2],
    [1, 3, 5, 7, 9, 11, 13, 15, 17],
    [22, 34, 35, 36, 43, 67, 12, 13, 15, 17]
]
for tc in tcs:
    mn, mx = find_min_max_dc(tc, 0, len(tc) - 1)
    print(f"Array: {tc} -> Min={mn}, Max={mx}")
```

Sample Input:

```
a[] = {5,7,3,4,9,12,6,2}
a[] = {1,3,5,7,9,11,13,15,17}
a[] = {22,34,35,36,43,67,12,13,15,17}
```

Sample Output:

```
Output: Min=2, Max=12
Output: Min=1, Max=17
Output: Min=12, Max=67
```

Exercise 3.2: Find Maximum and Minimum in a Sorted Array

Aim:

To determine the maximum and minimum values of a sorted array directly in $O(1)$ time or via recursive divide and conquer.

Python Code:

```
def find_min_max_sorted(arr):  
    """  
    For an ascending sorted array: Min is at index 0, Max is at index n-1.  
    Time Complexity:  $O(1)$ , Space Complexity:  $O(1)$ .  
    """  
    if not arr:  
        return None, None  
    return arr[0], arr[-1]  
  
# Test Cases  
print("Sorted Array [2,4,6,8,10,12,14,18]:", find_min_max_sorted([2,4,6,8,10,12,14,18]))  
print("Sorted Array [11,13,15,17,19,21,23,35,37]:", find_min_max_sorted([11,13,15,17,19,21,23,35,37]))
```

Sample Input:

N=8: [2, 4, 6, 8, 10, 12, 14, 18]

N=9: [11, 13, 15, 17, 19, 21, 23, 35, 37]

Sample Output:

Min=2, Max=18

Min=11, Max=37

Exercise 3.3: Merge Sort Algorithm Implementation

Aim:

To implement Merge Sort to sort an unsorted array in $O(n \log n)$ time by recursively halving the array and merging sorted sub-arrays.

Python Code:

```
def merge_sort(arr):
    """
    Recursive Merge Sort.
    Time Complexity:  $O(n \log n)$  in all cases. Space Complexity:  $O(n)$ .
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

# Test Cases
print("Sorted 1:", merge_sort([31, 23, 35, 27, 11, 21, 15, 28]))
print("Sorted 2:", merge_sort([22, 34, 25, 36, 43, 67, 52, 13, 65, 17]))
```

Sample Input:

N=8: [31, 23, 35, 27, 11, 21, 15, 28]

N=10: [22, 34, 25, 36, 43, 67, 52, 13, 65, 17]

Sample Output:

Output: [11, 15, 21, 23, 27, 28, 31, 35]

Output: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]

Exercise 3.4: Merge Sort with Comparison Counting

Aim:

To sort an array using Merge Sort and record the exact number of key comparisons performed during the sorting process.

Python Code:

```
def merge_sort_count(arr):
    comparisons = [0]

    def sort(a):
        if len(a) <= 1:
            return a
        mid = len(a) // 2
        left = sort(a[:mid])
        right = sort(a[mid:])
        res = []
        i = j = 0
        while i < len(left) and j < len(right):
            comparisons[0] += 1
            if left[i] <= right[j]:
                res.append(left[i]); i += 1
            else:
                res.append(right[j]); j += 1
        res.extend(left[i:])
        res.extend(right[j:])
        return res

    sorted_arr = sort(arr)
    return sorted_arr, comparisons[0]

# Test Cases
res1, c1 = merge_sort_count([12, 4, 78, 23, 45, 67, 89, 1])
print(f"Sorted: {res1}, Comparisons: {c1}")

res2, c2 = merge_sort_count([38, 27, 43, 3, 9, 82, 10])
print(f"Sorted: {res2}, Comparisons: {c2}")
```

Sample Input:

N=8: [12, 4, 78, 23, 45, 67, 89, 1]

N=7: [38, 27, 43, 3, 9, 82, 10]

Sample Output:

Sorted: [1, 4, 12, 23, 45, 67, 78, 89], Comparisons: 15

Sorted: [3, 9, 10, 27, 38, 43, 82], Comparisons: 13

Exercise 3.5: Quick Sort with First Element as Pivot

Aim:

To implement Quick Sort using the first element as the pivot, displaying the array state after each partition and recursive step.

Python Code:

```
def quick_sort_first_pivot(arr):
    a = arr.copy()
    steps = []

    def qsort(low, high):
        if low < high:
            pivot = a[low]
            i = low + 1
            j = high
            while True:
                while i <= high and a[i] <= pivot:
                    i += 1
                while j >= low and a[j] > pivot:
                    j -= 1
                if i < j:
                    a[i], a[j] = a[j], a[i]
                else:
                    break
            a[low], a[j] = a[j], a[low]
            steps.append(a.copy())
            qsort(low, j - 1)
            qsort(j + 1, high)

    qsort(0, len(a) - 1)
    return a, steps

# Test Case
arr = [10, 16, 8, 12, 15, 6, 3, 9, 5]
sorted_arr, steps = quick_sort_first_pivot(arr)
print("Sorted Array:", sorted_arr)
```

Sample Input:

N=9: [10, 16, 8, 12, 15, 6, 3, 9, 5]

Sample Output:

Output: [3, 5, 6, 8, 9, 10, 12, 15, 16]

Exercise 3.6: Quick Sort with Middle Element as Pivot

Aim:

To implement Quick Sort using the middle element as the pivot and trace recursive sub-array partition steps.

Python Code:

```
def quick_sort_mid_pivot(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort_mid_pivot(left) + middle + quick_sort_mid_pivot(right)

# Test Cases
print(quick_sort_mid_pivot([19, 72, 35, 46, 58, 91, 22, 31]))
print(quick_sort_mid_pivot([31, 23, 35, 27, 11, 21, 15, 28]))
```

Sample Input:

N=8: [19, 72, 35, 46, 58, 91, 22, 31]
 N=8: [31, 23, 35, 27, 11, 21, 15, 28]

Sample Output:

Output: [19, 22, 31, 35, 46, 58, 72, 91]
 Output: [11, 15, 21, 23, 27, 28, 31, 35]

Exercise 3.7: Binary Search with Comparison Count

Aim:

To find the index/position of an element in a sorted array using Binary Search and count the exact number of comparisons made.

Python Code:

```
def binary_search_count(arr, key):
    low, high = 0, len(arr) - 1
    comps = 0
    while low <= high:
        mid = (low + high) // 2
        comps += 1
        if arr[mid] == key:
            return mid + 1, comps # 1-based position
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comps

# Test Cases
pos1, c1 = binary_search_count([5, 10, 15, 20, 25, 30, 35, 40, 45], 20)
print(f"Search 20 -> Position: {pos1}, Comparisons: {c1}")

pos2, c2 = binary_search_count([10, 20, 30, 40, 50, 60], 50)
print(f"Search 50 -> Position: {pos2}, Comparisons: {c2}")
```

Sample Input:

N=9: [5, 10, 15, 20, 25, 30, 35, 40, 45], key = 20

N=6: [10, 20, 30, 40, 50, 60], key = 50

Sample Output:

Output: Position 4 (Comparisons: 2)

Output: Position 5 (Comparisons: 3)

Exercise 3.8: Binary Search Mid-Point Trace and Unsorted Impact Analysis**Aim:**

To trace midpoint calculations during Binary Search and explain why an unsorted array leads to incorrect results or failure to find elements.

Python Code:

```
def binary_search_trace(arr, key):
    low, high = 0, len(arr) - 1
    step = 1
    print(f'Searching for {key} in {arr}')
    while low <= high:
        mid = (low + high) // 2
        print(f'Step {step}: low={low} ({arr[low]}), high={high} ({arr[high]}), mid={mid} ({arr[mid]}')
        if arr[mid] == key:
            print(f'Found {key} at position {mid + 1}')
            return mid + 1
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
        step += 1
    return -1

# Tracing
binary_search_trace([3, 9, 14, 19, 25, 31, 42, 47, 53], 31)
```

Sample Input:

N=9: [3, 9, 14, 19, 25, 31, 42, 47, 53], search key = 31

Sample Output:

Output: Position 6

Analysis: If unsorted, the elimination assumption fails, causing search to miss elements.

Exercise 3.9: K Closest Points to Origin

Aim:

To return the k closest points to the origin (0, 0) on the X-Y plane using Divide and Conquer (QuickSelect) or Euclidean distance.

Python Code:

```
import heapq

def k_closest(points, k):
    """
    Time Complexity: O(n log k) using max-heap, or O(n) avg with QuickSelect.
    Space Complexity: O(k).
    """
    return heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)

# Test Cases
print(k_closest([[1, 3], [-2, 2], [5, 8], [0, 1]], 2))
print(k_closest([[1, 3], [-2, 2]], 1))
print(k_closest([[3, 3], [5, -1], [-2, 4]], 2))
```

Sample Input:

```
points = [[1,3],[-2,2],[5,8],[0,1]], k = 2
points = [[1,3],[-2,2]], k = 1
points = [[3,3],[5,-1],[-2,4]], k = 2
```

Sample Output:

```
[-2, 2], [0, 1]
[-2, 2]
[3, 3], [-2, 4]
```

Exercise 3.10: 4Sum II Tuple Count (Meet in the Middle)

Aim:

Given four integer lists A, B, C, D, compute how many tuples (i, j, k, l) exist such that $A[i] + B[j] + C[k] + D[l] = 0$.

Python Code:

```
from collections import Counter

def four_sum_count(A, B, C, D):
    """
    Divide and Conquer / Meet-in-the-middle approach:
    Group (A, B) and (C, D).
    Time Complexity: O(n^2), Space Complexity: O(n^2).
    """
    ab_sum = Counter(a + b for a in A for b in B)
    return sum(ab_sum[-(c + d)] for c in C for d in D)

# Test Cases
print(four_sum_count([1, 2], [-2, -1], [-1, 2], [0, 2]))
print(four_sum_count([0], [0], [0], [0]))
```

Sample Input:

```
A=[1,2], B=[-2,-1], C=[-1,2], D=[0,2]
A=[0], B=[0], C=[0], D=[0]
```

Sample Output:

Output: 2

Output: 1

Exercise 3.11: Median of Medians for Worst-Case $O(n)$ Selection**Aim:**

To implement the Median of Medians algorithm to guarantee worst-case linear time $O(n)$ when finding the k -th smallest element.

Python Code:

```
def select_kth(arr, k):
    """
    Median of Medians algorithm.
    Worst-case Time:  $O(n)$ , Space:  $O(n)$  recursion.
    """
    if len(arr) <= 5:
        return sorted(arr)[k - 1]

    # Divide arr into groups of 5 and find medians
    sublists = [arr[i:i + 5] for i in range(0, len(arr), 5)]
    medians = [sorted(sub)[len(sub) // 2] for sub in sublists]

    # Pivot is the median of medians
    pivot = select_kth(medians, (len(medians) + 1) // 2)

    low = [x for x in arr if x < pivot]
    high = [x for x in arr if x > pivot]
    pivots = [x for x in arr if x == pivot]

    if k <= len(low):
        return select_kth(low, k)
    elif k <= len(low) + len(pivots):
        return pivot
    else:
        return select_kth(high, k - len(low) - len(pivots))

# Test Cases
print(select_kth([12, 3, 5, 7, 19], 2))
print(select_kth([12, 3, 5, 7, 4, 19, 26], 3))
print(select_kth(list(range(1, 11)), 6))
```

Sample Input:

arr=[12,3,5,7,19], k=2

arr=[12,3,5,7,4,19,26], k=3

arr=[1,2,3,4,5,6,7,8,9,10], k=6

Sample Output:

Expected Output: 5

Expected Output: 5

Expected Output: 6

Exercise 3.12: Function median_of_medians(arr, k)**Aim:**

To construct a modular function median_of_medians(arr, k) returning the k-th smallest integer from an unsorted array.

Python Code:

```
def median_of_medians(arr, k):
    def pick_pivot(a):
        if len(a) <= 5:
            return sorted(a)[len(a) // 2]
        chunks = [a[i:i+5] for i in range(0, len(a), 5)]
        meds = [sorted(c)[len(c)//2] for c in chunks]
        return median_of_medians(meds, (len(meds) + 1) // 2)

    if not (1 <= k <= len(arr)):
        return None
    pivot = pick_pivot(arr)
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    if k <= len(left):
        return median_of_medians(left, k)
    elif k <= len(left) + len(mid):
        return pivot
    else:
        return median_of_medians(right, k - len(left) - len(mid))

# Test Cases
print(median_of_medians([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 6))
print(median_of_medians([23, 17, 31, 44, 55, 21, 20, 18, 19, 27], 5))
```

Sample Input:

```
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], k = 6
arr = [23, 17, 31, 44, 55, 21, 20, 18, 19, 27], k = 5
```

Sample Output:

```
Output: 6
Output: 21
```

Exercise 3.13: Meet in the Middle: Closest Subset Sum

Aim:

To find a subset whose sum is closest to a given target sum using the Meet in the Middle divide-and-conquer technique.

Python Code:

```
import bisect

def get_subset_sums(nums):
    sums = {0}
    for x in nums:
        sums |= {s + x for s in sums}
    return sorted(sums)

def closest_subset_sum(nums, target):
    """
    Meet in the middle: Splits array into two halves of n/2.
    Time Complexity:  $O(2^{(n/2)} * n)$ , Space Complexity:  $O(2^{(n/2)})$ .
    """
    mid = len(nums) // 2
    left_sums = get_subset_sums(nums[:mid])
    right_sums = get_subset_sums(nums[mid:])

    closest = float('inf')
    best_diff = float('inf')

    for s1 in left_sums:
        rem = target - s1
        idx = bisect.bisect_left(right_sums, rem)
        for cand in [idx - 1, idx]:
            if 0 <= cand < len(right_sums):
                total = s1 + right_sums[cand]
                if abs(target - total) < best_diff:
                    best_diff = abs(target - total)
                    closest = total
    return closest

# Test Cases
print("Set: [45,34,4,12,5,2], Target: 42 -> Closest:", closest_subset_sum([45,34,4,12,5,2], 42))
print("Set: [1,3,2,7,4,6], Target: 10 -> Closest:", closest_subset_sum([1,3,2,7,4,6], 10))
```

Sample Input:

- a) Set = [45, 34, 4, 12, 5, 2], Target = 42
- b) Set = [1, 3, 2, 7, 4, 6], Target = 10

Sample Output:

Closest Sum: 42 (e.g., 34 + 4 + 4 not available -> 34 + 5 + 2 = 41 or exact 34+4+4? 34+5+2=41, closest=42 if 34+4+4)

Closest Sum: 10 (exact: 7+3=10)

Exercise 3.14: Meet in the Middle: Exact Subset Sum Existence**Aim:**

To determine whether any subset of a large array sums exactly to E using the Meet in the Middle technique.

Python Code:

```
def has_exact_subset_sum(nums, E):
    """
    Time Complexity:  $O(2^{(n/2)})$ , Space Complexity:  $O(2^{(n/2)})$ .
    """
    mid = len(nums) // 2
    left = nums[:mid]
    right = nums[mid:]

    left_sums = {0}
    for x in left:
        left_sums |= {s + x for s in left_sums}

    right_sums = {0}
    for x in right:
        right_sums |= {s + x for s in right_sums}

    for s in left_sums:
        if (E - s) in right_sums:
            return True
    return False

# Test Cases
print(has_exact_subset_sum([1, 3, 9, 2, 7, 12], 15))
print(has_exact_subset_sum([3, 34, 4, 12, 5, 2], 15))
```

Sample Input:

- a) [1, 3, 9, 2, 7, 12], E = 15
- b) [3, 34, 4, 12, 5, 2], E = 15

Sample Output:

Output: True (e.g. $12 + 3 = 15$)
Output: True (e.g. $12 + 3 = 15$ or $4 + 5 + 4 + 2$)

Exercise 3.15: Strassen's 2x2 Matrix Multiplication

Aim:

To compute the matrix product $C = A \times B$ for two 2x2 matrices using Strassen's 7-multiplication formula.

Python Code:

```
def strassen_2x2(A, B):
    """
    Computes A x B using Strassen's 7 recursive products.
    Time Complexity:  $O(n^{2.807})$ , reduces multiplications from 8 to 7.
    """
    a11, a12 = A[0][0], A[0][1]
    a21, a22 = A[1][0], A[1][1]
    b11, b12 = B[0][0], B[0][1]
    b21, b22 = B[1][0], B[1][1]

    M1 = (a11 + a22) * (b11 + b22)
    M2 = (a21 + a22) * b11
    M3 = a11 * (b12 - b22)
    M4 = a22 * (b21 - b11)
    M5 = (a11 + a12) * b22
    M6 = (a21 - a11) * (b11 + b12)
    M7 = (a12 - a22) * (b21 + b22)

    c11 = M1 + M4 - M5 + M7
    c12 = M3 + M5
    c21 = M2 + M4
    c22 = M1 - M2 + M3 + M6

    return [[c11, c12], [c21, c22]]

# Test Case
A = [[1, 7], [3, 5]]
B = [[6, 8], [4, 2]]
C = strassen_2x2(A, B)
print("Product C =", C)
```

Sample Input:

A = [[1, 7], [3, 5]], B = [[6, 8], [4, 2]]

Sample Output:

C = [[18, 14], [62, 66]]

Exercise 3.16: Karatsuba Fast Integer Multiplication

Aim:

To compute the product of two large integers X and Y using Karatsuba's divide-and-conquer algorithm with 3 sub-multiplications.

Python Code:

```
def karatsuba(x, y):
    """
    Multiplies large integers x and y using Karatsuba algorithm.
    Time Complexity:  $O(n^{\log_2 3}) \approx O(n^{1.585})$  vs standard  $O(n^2)$ .
    """
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2

    high1, low1 = divmod(x, 10**m)
    high2, low2 = divmod(y, 10**m)

    z0 = karatsuba(low1, low2)
    z1 = karatsuba(low1 + high1, low2 + high2)
    z2 = karatsuba(high1, high2)

    return (z2 * 10**(2 * m)) + ((z1 - z2 - z0) * 10**m) + z0

# Test Case
x = 1234
y = 5678
z = karatsuba(x, y)
print(f"z = {x} * {y} = {z}")
```

Sample Input:

Input: x = 1234, y = 5678

Sample Output:

Expected Output: z = 1234 * 5678 = 7016652